Medumba Flutter Mobile App - Architecture

Date: March 25, 2026 | **Project:** Medumba Mobile (Flutter) | **Platforms:** iOS & Android

---

Architecture Overview

The Medumba Flutter app uses a **clean, layered architecture** with 6 key layers:

```
┌─────────────────────────────────────────────────────────┐
│ PRESENTATION: Screens (Auth, Onboarding, Dashboard)     │
├─────────────────────────────────────────────────────────┤
│ STATE MANAGEMENT: Riverpod Providers                    │
├─────────────────────────────────────────────────────────┤
│ SERVICES: Auth, Gamification, Lessons, Audio            │
├─────────────────────────────────────────────────────────┤
│ REPOSITORIES: Data abstraction layer                    │
├─────────────────────────────────────────────────────────┤
│ LOCAL STORAGE: SQLite + Secure Storage                  │
├─────────────────────────────────────────────────────────┤
│ REMOTE SERVICES: Backend API + Firebase                 │
└─────────────────────────────────────────────────────────┘
```

Layer Breakdown

1. **Presentation Layer**

- **Auth Screens:** Splash, Welcome, Login, Signup, Password Reset
- **Onboarding:** Language selection, Proficiency level, Goals setup
- **Main App:** Dashboard, Lessons, Counting, Dictionary, Video, Calendar
- **Tech:** Flutter Material 3, GoRouter (navigation)

2. **State Management Layer**

- **Riverpod Providers:**
  - `AuthProvider` - Authentication state & sessions
  - `UserProvider` - Profile, stats, preferences
  - `LessonProvider` - Current/available lessons
  - `ProgressProvider` - XP, streaks, achievements
- **Why Riverpod:** Simpler than BLoC, better testability, less boilerplate

3. **Business Logic (Services)**

Service	Responsibilities
AuthService	Login, signup, OTP verification, password reset

Service	Responsibilities
GameService	XP calculation, streak tracking, achievements
LessonService	Lesson generation, progress tracking, caching
AudioService	Playback, recording, decoding (Opus/OGG)
DataService	Dictionary, expressions, video content

4. **Repository Layer**

- Abstracts data sources behind interfaces
- Three main repos: `AuthRepository`, `UserDataRepository`, `LessonRepository`
- Enables easy testing with mock implementations

5. **Local Storage Layer**

Storage	Purpose	Package
Secure Storage	API tokens, credentials	`flutter_secure_storage`
SQLite	Users, progress, lessons, stats	`sqflite`
File Cache	Videos, audio, images	`path_provider`

6. **Remote Services Layer**

- **Backend API:** Authentication, user data, lessons, analytics
- **Firebase:** Email/OTP, crash reporting, analytics
- **HTTP Client:** `dio` with interceptors, retry logic, offline detection

---

Project Structure

```
lib/
├── presentation/
│   ├── screens/              # Auth, onboarding, dashboard pages
│   ├── widgets/              # Reusable UI components
│   └── providers/            # Riverpod state management
├── domain/
│   ├── models/              # Data models (user, lesson, progress)
│   └── repositories/        # Abstract interfaces
├── data/
│   ├── datasources/         # Remote API & local DB access
│   └── repositories/        # Concrete implementations
├── services/                # Business logic services
├── utils/                   # Helpers, validators, logging
└── generated/               # i18n & build-generated files
```

Technology Stack

Layer	Technology
Framework	Flutter 3.19+
Language	Dart 3.2+
State Mgmt	Riverpod 2.4+
Routing	GoRouter 11.0+
HTTP	Dio 5.3+
Database	SQLite (sqflite)
Secure Storage	flutter_secure_storage
Localization	intl + flutter_localizations
Audio	just_audio 0.9+
Video	chewie 1.7+
Analytics	Firebase Analytics
Error Tracking	Firebase Crashlytics

Advantages of Using Flutter for Mobile

🚀 1. Cross-Platform Development (Single Codebase)

- **Code Reuse:** 95%+ of code shared between iOS & Android
- **Faster Development:** Build for both platforms simultaneously
- **Faster Updates:** One deployment pipeline for both stores
- **Cost Savings:** Single team maintains one codebase instead of two

📱 2. Performance

- **Native-Level Speed:** Compiles to native ARM code (not interpreted)
- **High FPS:** 60-120 FPS animations out-of-the-box
- **Minimal Overhead:** No JavaScript bridge (unlike React Native)
- **Efficient:** Lower battery drain and better RAM usage

🎨 3. Rich UI/UX

- **Material 3 & Cupertino:** Built-in design systems for both platforms
- **Customizable Widgets:** Complete control over appearance
- **Animations:** Smooth, performant animations by default
- **Hot Reload:** Real-time code changes during development

🛠️ 4. Developer Experience

- **Dart Language:** Modern, easy to learn (similar to Java/Swift)

- **Hot Reload:** Instant feedback without recompiling
- **Hot Restart:** Rebuild app state in <1 second
- **Excellent Documentation:** Official docs + large community
- **Great IDE Support:** VS Code, Android Studio plugins

🔧 5. Offline-First Architecture

- **SQLite Integration:** Built-in database support
- **Caching:** Easy media caching for offline access
- **Reliable:** Tested extensively in production apps
- **Perfect for Language Learning:** Content can be downloaded and used offline

🔐 6. Security

- **Secure Storage:** OS-level encryption (Keychain/Keystore)
- **Firebase Integration:** Authentication + Crash monitoring
- **Minimal Attack Surface:** No web view vulnerabilities (unlike some frameworks)
- **Regular Security Updates:** Flutter team patches actively

📊 7. App Size & Performance

- **Smaller APK:** ~40-80 MB for a basic app (vs. 100+ MB for React Native)
- **Startup Time:** Cold start <2 seconds
- **RAM Usage:** ~60-100 MB runtime (efficient)
- **Battery:** Minimal drain due to native compilation

🧪 8. Testing & Quality

- **Unit Testing:** Native support with `test` package
- **Widget Testing:** Easy UI component testing
- **Integration Testing:** Full app flow testing
- **CI/CD:** Easy integration with GitHub Actions, Fastlane

💡 9. Gamification Features (For Medumba)

- **Real-Time Animations:** Smooth reward animations, progress bars
- **Audio/Video:** First-class support for multimedia
- **Responsive Design:** Handles all screen sizes elegantly
- **Platform-Specific Features:** Native notifications, haptics

🌐 10. Web & Desktop Extension

- **Code Reuse:** Same codebase can target web & desktop
- **Firebase Integration:** Scales from mobile to multi-platform
- **Single Backend:** One API for all platforms

Key Advantages for Medumba Specifically

- ☑ **Linguistics Features:** Easy implementation of audio playback, recording, and pronunciation analysis
- ☑ **Offline Learning:** Users can download lessons and access without internet
- ☑ **Gamification:** Smooth animations for rewards, streaks, achievements
- ☑ **Multi-Language Support:** Native i18n support scales well
- ☑ **Fast Iteration:** Hot reload makes A/B testing and rapid updates easy
- ☑ **Cost Effective:** One team, one codebase for iOS + Android
- ☑ **Scalability:** Architecture supports millions of concurrent users

---

Performance Targets

- **App Startup:** < 3 seconds
- **Screen Transitions:** 60 FPS (consistent)
- **API Response:** < 2 seconds
- **Database Query:** < 100 ms
- **APK Size:** 50-80 MB
- **RAM Usage:** 80-120 MB

Security Best Practices

✓ Tokens stored in secure storage (OS-level encryption)
✓ HTTPS-only communication
✓ Firebase Authentication for OTP/email verification
✓ Firebase Crashlytics for error monitoring
✓ No sensitive data in SharedPreferences
✓ Build signing for Play Store & App Store

Migration from React Web App

Transferable:

- Authentication flows
- Lesson content & data structures
- Gamification logic
- User profile schema
- Dictionary & expressions

Changed:

- UI components → Flutter widgets
- React Router → GoRouter
- State management → Riverpod
- localStorage → SQLite + Secure Storage

Document Version: 1.0
Status: Approved Architecture Proposal